STATS 506 - Homework 3

GitHub repository: https://github.com/wei-min-lin/Homework1

```
library(haven)
library(DBI)
library(RSQLite)
library(microbenchmark)
library(knitr)

options(width = 76, digits = 5)
set.seed(506)
dir.create("data", showWarnings = FALSE)

get_file <- function(url, path) {
  if (!file.exists(path)) {
    download.file(url, path, mode = "wb", quiet = TRUE)
  }
  path
}
```

Problem 1

1. Import and merge

```
nhanes_url <- paste0(
  "https://wwwn.cdc.gov/Nchs/Data/Nhanes/Public/2015/DataFiles/"
)
aux <- as.data.frame(read_xpt(get_file(
  paste0(nhanes_url, "AUX_I.xpt"), "data/AUX_I.xpt"
)))
demo <- as.data.frame(read_xpt(get_file(
  paste0(nhanes_url, "DEMO_I.xpt"), "data/DEMO_I.xpt"
)))
stopifnot(!anyDuplicated(aux$SEQN), !anyDuplicated(demo$SEQN))
nhanes <- merge(aux, demo, by = "SEQN", all = FALSE)
dim(nhanes)
```

```
[1] 4582  119
```

2. Clean variables

Income codes 12 and 13 overlap the other income ranges, so they cannot be placed in a single increasing sequence. I retain them in the labeled factor but exclude them from the numeric income variable. The regression uses interval midpoints in units of USD 10,000; I assign USD 125,000 to the open-ended top category. The children variable is top-coded at 3, meaning three or more.

```
nhanes$gender <- factor(
  nhanes$RIAGENDR, levels = c(1, 2), labels = c("Male", "Female")
)
```

```r
nhanes$citizenship <- factor(
  nhanes$DMDCITZN, levels = c(1, 2),
  labels = c("Citizen", "Noncitizen")
)
nhanes$children <- ifelse(
  nhanes$DMDHHSZA %in% 0:3, nhanes$DMDHHSZA, NA_real_
)
income_codes <- c(1:10, 14, 15, 13, 12)
income_labels <- c(
  "0-4,999", "5,000-9,999", "10,000-14,999", "15,000-19,999",
  "20,000-24,999", "25,000-34,999", "35,000-44,999",
  "45,000-54,999", "55,000-64,999", "65,000-74,999",
  "75,000-99,999", "100,000+", "Under 20,000 (broad)",
  "20,000+ (broad)"
)
nhanes$income_category <- factor(
  nhanes$INDHHIN2, levels = income_codes, labels = income_labels
)
midpoints <- c(
  2500, 7500, 12500, 17500, 22500, 30000,
  40000, 50000, 60000, 70000, 87500, 125000
)
nhanes$income <- midpoints[
  match(nhanes$INDHHIN2, c(1:10, 14, 15))
] / 10000
nhanes$right <- ifelse(
  nhanes$AUXTWIDR %in% c(555, 777, 888), NA, nhanes$AUXTWIDR
)
nhanes$left <- ifelse(
  nhanes$AUXTWIDL %in% c(555, 777, 888), NA, nhanes$AUXTWIDL
)
vars <- c("gender", "citizenship", "children", "income", "right", "left")
kable(data.frame(
  Variable = vars, Missing = unname(colSums(is.na(nhanes[vars])))
), caption = "Missing values after cleaning")
```

Table 1: Missing values after cleaning

Variable	Missing
gender	0
citizenship	14
children	0
income	495
right	433
left	479

3. Poisson models

I fit the requested unweighted Poisson models with a log link. IRRs are exponentiated slopes; the exponentiated intercept is the baseline fitted mean. McFadden's pseudo-R^2 is $1 - \ell(\hat{\beta})/\ell(\hat{\beta}_0)$, with the null model fitted to the same observations.

```r
models <- list(
  "1R" = glm(right ~ gender, data = nhanes, family = poisson()),
  "2R" = glm(right ~ gender + citizenship + children + income,
             data = nhanes, family = poisson()),
```

```
  "1L" = glm(left ~ gender, data = nhanes, family = poisson()),
  "2L" = glm(left ~ gender + citizenship + children + income,
             data = nhanes, family = poisson())
)
term_names <- c(
  "(Intercept)", "genderFemale", "citizenshipNoncitizen",
  "children", "income"
)
irr <- data.frame(Term = c(
  "Baseline mean", "Female vs. male", "Noncitizen vs. citizen",
  "Children (per unit)", "Income (per USD 10,000)"
))
for (name in names(models)) {
  irr[[name]] <- exp(coef(models[[name]])[term_names])
}
options(knitr.kable.NA = "--")
kable(irr, digits = 3, caption = "IRRs and baseline fitted means")
```

Table 2: IRRs and baseline fitted means

Term	1R	2R	1L	2L
Baseline mean	84.317	86.725	84.758	86.818
Female vs. male	1.009	1.015	1.013	1.019
Noncitizen vs. citizen	–	1.046	–	1.022
Children (per unit)	–	0.994	–	0.983
Income (per USD 10,000)	–	0.993	–	0.995

```
model_stats <- data.frame()
for (name in names(models)) {
  fit <- models[[name]]
  response <- model.response(model.frame(fit))
  null <- glm(response ~ 1, family = poisson())
  model_stats <- rbind(model_stats, data.frame(
    Model = name, N = nobs(fit),
    McFadden_R2 = 1 - as.numeric(logLik(fit) / logLik(null)),
    AIC = AIC(fit),
    Dispersion = sum(residuals(fit, type = "pearson")^2) /
      df.residual(fit)
  ))
}
kable(model_stats, digits = c(0, 0, 4, 1, 2),
      caption = "Model statistics")
```

Table 3: Model statistics

Model	N	McFadden_R2	AIC	Dispersion
1R	4149	0.0001	96618	20.90
2R	3705	0.0047	84868	20.32
1L	4103	0.0001	98685	22.56
2L	3665	0.0024	86619	22.01

The dispersion estimates compare Pearson residual variation with the Poisson assumption of equal mean and variance. Values well above 1 indicate overdispersion. These models also do not account for the NHANES survey design, so the tests below are model-based, not nationally representative survey inference.

4. Sex comparisons in model 2L

First, I test $H_0 : \beta_{\text{Female}} = 0$, or equivalently, an IRR of 1.

```
fit <- models[["2L"]]
b <- coef(fit)["genderFemale"]
se_b <- sqrt(vcov(fit)["genderFemale", "genderFemale"])
z_irr <- unname(b / se_b)
p_irr <- 2 * pnorm(-abs(z_irr))
irr_test <- data.frame(
  IRR = exp(unname(b)),
  Lower_95 = exp(unname(b) - 1.96 * se_b),
  Upper_95 = exp(unname(b) + 1.96 * se_b),
  z = z_irr, p_value = format.pval(p_irr, digits = 3, eps = 0.001)
)
kable(irr_test, digits = 3, caption = "Female-to-male IRR")
```

Table 4: Female-to-male IRR

IRR	Lower_95	Upper_95	z	p_value
1.019	1.012	1.026	5.215	<0.001

The female-to-male IRR is 1.019, corresponding to a 1.9% higher expected left-ear width at the same covariate values. The Wald test rejects the null at the 5% level.

Next, I predict each complete-case respondent twice, once as male and once as female, while keeping the other covariates unchanged. I test whether the difference between these two average predictions is zero. The delta-method standard error includes the covariance between the predictions.

```
new_male <- new_female <- model.frame(fit)
new_male$gender <- factor("Male", levels = levels(nhanes$gender))
new_female$gender <- factor("Female", levels = levels(nhanes$gender))
x_male <- model.matrix(delete.response(terms(fit)), new_male)
x_female <- model.matrix(delete.response(terms(fit)), new_female)
mu_male <- as.vector(exp(x_male %*% coef(fit)))
mu_female <- as.vector(exp(x_female %*% coef(fit)))
difference <- mean(mu_female - mu_male)
gradient <- colMeans(x_female * mu_female - x_male * mu_male)
se_difference <- sqrt(drop(t(gradient) %*% vcov(fit) %*% gradient))
z_difference <- difference / se_difference
p_difference <- 2 * pnorm(-abs(z_difference))
kable(data.frame(
  Male = mean(mu_male), Female = mean(mu_female),
  Difference = difference, SE = se_difference,
  Lower_95 = difference - 1.96 * se_difference,
  Upper_95 = difference + 1.96 * se_difference
), digits = 3, caption = "Adjusted predicted means and female-minus-male contrast")
```

Table 5: Adjusted predicted means and female-minus-male contrast

Male	Female	Difference	SE	Lower_95	Upper_95
84.202	85.798	1.596	0.306	0.996	2.195

```
kable(data.frame(
  z = z_difference,
```

```
  p_value = format.pval(p_difference, digits = 3, eps = 0.001)
), digits = 3, caption = "Test of equal adjusted predicted means")
```

Table 6: Test of equal adjusted predicted means

z	p_value
5.218	<0.001

The adjusted means differ by 1.596 units (female minus male). The test rejects equal predicted means at the 5% level. With no sex interactions, both tests concern the same underlying null, although the Wald statistics use different scales.

Problem 2

Each R approach includes the SQL extraction and subsequent R calculations. Each SQL approach uses one query. The benchmarks exclude downloads, connection setup, and printing, and include data retrieval in both methods. No `_list` tables are used.

```
db_url <- paste0(
  "https://raw.githubusercontent.com/bradleygrant/",
  "sakila-sqlite3/main/sakila_master.db"
)
get_file(db_url, "data/sakila_master.db")
```

```
[1] "data/sakila_master.db"
```

```
con <- dbConnect(SQLite(), "data/sakila_master.db")

benchmark_table <- function(result) {
  out <- summary(result, unit = "ms")
  kable(out[c("expr", "min", "median", "mean", "max", "neval")],
        digits = 3, caption = "Runtime in milliseconds")
}
```

1. Customers by store

```
customers_r <- function(con) {
  customers <- dbGetQuery(con,
    "SELECT customer_id, store_id, active FROM customer")
  customers$active <- as.integer(
    as.character(customers$active) %in% c("1", "Y")
  )
  counts <- aggregate(customer_id ~ store_id, customers, length)
  names(counts)[2] <- "customers"
  active <- aggregate(active ~ store_id, customers, mean)
  active$active_percent <- 100 * active$active
  merge(counts, active[c("store_id", "active_percent")],
        by = "store_id")
}
customer_query <- "
SELECT store_id, COUNT(*) AS customers,
       100.0 * AVG(CASE WHEN active IN (1, 'Y') THEN 1.0
                        ELSE 0.0 END) AS active_percent
FROM customer
GROUP BY store_id
```

```
ORDER BY store_id
"
customers_sql <- dbGetQuery(con, customer_query)
stopifnot(isTRUE(all.equal(customers_r(con), customers_sql,
                           check.attributes = FALSE)))
kable(customers_sql, digits = 2,
      caption = "Customer counts and active percentages")
```

Table 7: Customer counts and active percentages

store_id	customers	active_percent
1	326	97.55
2	273	97.44

```
bench_customers <- microbenchmark(
  R = customers_r(con), SQL = dbGetQuery(con, customer_query),
  times = 50L
)
benchmark_table(bench_customers)
```

Table 8: Runtime in milliseconds

expr	min	median	mean	max	neval
R	1.806	2.028	2.159	6.199	50
SQL	0.310	0.389	0.398	0.571	50

2. Staff names and countries

I use each staff member's address, not the store's address.

```
staff_r <- function(con) {
  staff <- dbGetQuery(con,
    "SELECT staff_id, first_name, last_name, address_id FROM staff")
  address <- dbGetQuery(con, "SELECT address_id, city_id FROM address")
  city <- dbGetQuery(con, "SELECT city_id, country_id FROM city")
  country <- dbGetQuery(con, "SELECT country_id, country FROM country")
  out <- merge(staff, address, by = "address_id")
  out <- merge(out, city, by = "city_id")
  out <- merge(out, country, by = "country_id")
  out$name <- paste(out$first_name, out$last_name)
  out <- out[order(out$staff_id), c("staff_id", "name", "country")]
  rownames(out) <- NULL
  out
}
staff_query <- "
SELECT s.staff_id, s.first_name || ' ' || s.last_name AS name,
       co.country
FROM staff AS s
JOIN address AS a ON s.address_id = a.address_id
JOIN city AS ci ON a.city_id = ci.city_id
JOIN country AS co ON ci.country_id = co.country_id
ORDER BY s.staff_id
"
staff_sql <- dbGetQuery(con, staff_query)
```

```r
stopifnot(isTRUE(all.equal(staff_r(con), staff_sql,
                           check.attributes = FALSE)))
kable(staff_sql, caption = "Staff members and countries")
```

Table 9: Staff members and countries

staff_id	name	country
1	Mike Hillyer	Canada
2	Jon Stephens	Australia

```r
bench_staff <- microbenchmark(
  R = staff_r(con), SQL = dbGetQuery(con, staff_query), times = 50L
)
benchmark_table(bench_staff)
```

Table 10: Runtime in milliseconds

expr	min	median	mean	max	neval
R	1.997	2.169	2.256	3.344	50
SQL	0.216	0.283	0.354	3.291	50

3. Films with the highest rental revenue

I interpret highest dollar value as the total payments received for a film across all rentals and copies. Amounts are summed in cents to avoid floating-point comparisons, and all ties are retained.

```r
films_r <- function(con) {
  payment <- dbGetQuery(con, "SELECT rental_id, amount FROM payment")
  rental <- dbGetQuery(con,
    "SELECT rental_id, inventory_id FROM rental")
  inventory <- dbGetQuery(con,
    "SELECT inventory_id, film_id FROM inventory")
  film <- dbGetQuery(con, "SELECT film_id, title FROM film")
  out <- merge(payment, rental, by = "rental_id")
  out <- merge(out, inventory, by = "inventory_id")
  out <- merge(out, film, by = "film_id")
  out$cents <- round(100 * out$amount)
  totals <- aggregate(cents ~ film_id + title, out, sum)
  top <- totals[totals$cents == max(totals$cents), ]
  top$total_usd <- top$cents / 100
  top <- top[order(top$title), c("film_id", "title", "total_usd")]
  rownames(top) <- NULL
  top
}
film_query <- "
WITH totals AS (
  SELECT f.film_id, f.title,
         SUM(CAST(ROUND(100 * p.amount) AS INTEGER)) AS cents
  FROM payment AS p
  JOIN rental AS r ON p.rental_id = r.rental_id
  JOIN inventory AS i ON r.inventory_id = i.inventory_id
  JOIN film AS f ON i.film_id = f.film_id
  GROUP BY f.film_id, f.title
)
```

```
SELECT film_id, title, cents / 100.0 AS total_usd
FROM totals
WHERE cents = (SELECT MAX(cents) FROM totals)
ORDER BY title
"
films_sql <- dbGetQuery(con, film_query)
stopifnot(isTRUE(all.equal(films_r(con), films_sql,
                           check.attributes = FALSE)))
kable(films_sql, digits = 2, caption = "Highest total rental revenue")
```

Table 11: Highest total rental revenue

film_id	title	total_usd
879	TELEGRAPH VOYAGE	231.73

```
bench_films <- microbenchmark(
  R = films_r(con), SQL = dbGetQuery(con, film_query), times = 50L
)
benchmark_table(bench_films)
```

Table 12: Runtime in milliseconds

expr	min	median	mean	max	neval
R	45.252	47.768	48.770	60.290	50
SQL	31.146	32.939	33.383	40.585	50

```
dbDisconnect(con)
```

```
comparison <- data.frame()
benchmarks <- list(Customers = bench_customers, Staff = bench_staff,
                   Films = bench_films)
for (name in names(benchmarks)) {
  med <- tapply(benchmarks[[name]]$time,
                benchmarks[[name]]$expr, median) / 1e6
  comparison <- rbind(comparison, data.frame(
    Task = name, R_ms = unname(med["R"]),
    SQL_ms = unname(med["SQL"]),
    R_divided_by_SQL = unname(med["R"] / med["SQL"])
  ))
}
kable(comparison, digits = 3, caption = "Median runtime comparison")
```

Table 13: Median runtime comparison

Task	R_ms	SQL_ms	R_divided_by_SQL
Customers	2.028	0.389	5.215
Staff	2.169	0.283	7.671
Films	47.768	32.939	1.450

A ratio above 1 means SQL was faster. The two methods return the same answers; these timings describe this machine and database, rather than a universal speed difference.

Problem 3

```
au_zip <- get_file(
  "https://www.briandunning.com/sample-data/au-500.zip",
  "data/au-500.zip"
)
if (!file.exists("data/au-500.csv")) {
  unzip(au_zip, files = "au-500.csv", exdir = "data")
}
au <- read.csv("data/au-500.csv", colClasses = "character")
stopifnot(nrow(au) == 500)
```

1. Websites ending in .com

```
host <- tolower(sub("^[[:alpha:]]+://", "", trimws(au$web)))
host <- sub("[/:?#].*$", "", host)
is_com <- grepl("\\.com$", host)
com_percent <- 100 * mean(is_com)
kable(data.frame(Count = sum(is_com), Percent = com_percent),
      digits = 2, caption = "Websites ending in .com")
```

Table 14: Websites ending in .com

Count	Percent
0	0

0% end in `.com`. The end-of-string anchor excludes domains such as `.com.au`.

2. Most common email domain

```
domain <- tolower(sub("^.*@", "", trimws(au$email)))
domain_counts <- table(domain)
most_common <- domain_counts[domain_counts == max(domain_counts)]
kable(data.frame(Domain = names(most_common),
                 Count = as.integer(most_common)),
      caption = "Most common email domain(s)")
```

Table 15: Most common email domain(s)

Domain	Count
hotmail.com	114

3. Non-alphabetic characters in company names

```
nonalpha <- grepl("[^[:alpha:][:space:],]", au$company_name)
nonalpha_no_amp <- grepl("[^[:alpha:][:space:],&]", au$company_name)
kable(data.frame(
  Excluded = c("Commas and whitespace", "Commas, whitespace, ampersands"),
  Count = c(sum(nonalpha), sum(nonalpha_no_amp)),
  Proportion = c(mean(nonalpha), mean(nonalpha_no_amp))
), digits = 3, caption = "Company names containing other characters")
```

Table 16: Company names containing other characters

Excluded	Count	Proportion
Commas and whitespace	45	0.090
Commas, whitespace, ampersands	4	0.008

4. Phone formatting

```
format_phone <- function(x) {
  digits <- gsub("[^0-9]", "", x)
  stopifnot(all(nchar(digits) == 10))
  sub("^([0-9]{4})([0-9]{3})([0-9]{3})$", "\\1-\\2-\\3", digits)
}
au$phone1 <- format_phone(au$phone1)
au$phone2 <- format_phone(au$phone2)
kable(head(au[c("phone1", "phone2")], 10),
      caption = "First ten phone numbers in each column")
```

Table 17: First ten phone numbers in each column

phone1	phone2
0381-749-123	0458-665-290
0799-973-366	0497-622-620
0855-589-019	0427-885-282
0260-444-682	0443-795-912
0214-556-085	0453-666-885
0878-681-355	0451-966-921
0865-228-931	0427-991-688
0252-269-402	0415-961-606
0731-849-989	0411-732-965
0868-904-661	0461-862-457

5. Log apartment numbers

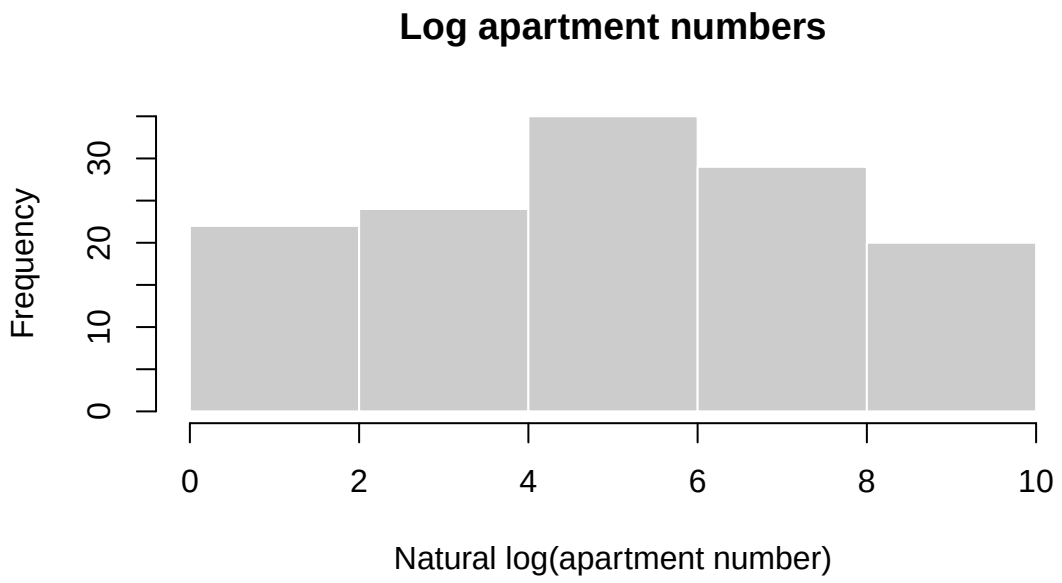
I extract digits at the end of an address and use the natural logarithm. Zero has no finite logarithm and is excluded from this plot.

```
has_apartment <- grepl("[0-9]+[[:space:]]*$", au$address)
apartment <- as.numeric(sub(
  ".*?([0-9]+)[[:space:]]*$", "\\1",
  au$address[has_apartment], perl = TRUE
))
positive_apartment <- apartment[apartment > 0]
kable(data.frame(
  Extracted = length(apartment),
  Zero = sum(apartment == 0),
  Positive = length(positive_apartment)
), caption = "Apartment numbers used in the analysis")
```

Table 18: Apartment numbers used in the analysis

Extracted	Zero	Positive
130	0	130

```r
hist(log(positive_apartment), breaks = "FD",
     main = "Log apartment numbers", xlab = "Natural log(apartment number)",
     col = "grey80", border = "white")
```



6. Benford's law

Under Benford's law, the leading digit has probabilities $P(D = d) = \log_{10}(1 + 1/d)$ for $d = 1, \ldots, 9$. I compare these probabilities with the positive apartment numbers using a chi-squared goodness-of-fit test.

```r
leading <- as.integer(substr(as.character(positive_apartment), 1, 1))
observed <- tabulate(leading, nbins = 9)
benford <- log10(1 + 1 / (1:9))
benford_test <- chisq.test(observed, p = benford)
kable(data.frame(
  Digit = 1:9, Count = observed,
  Observed = observed / sum(observed), Expected = benford,
  Expected_count = sum(observed) * benford
), digits = c(0, 0, 3, 3, 2), caption = "Leading digits versus Benford's law")
```

Table 19: Leading digits versus Benford's law

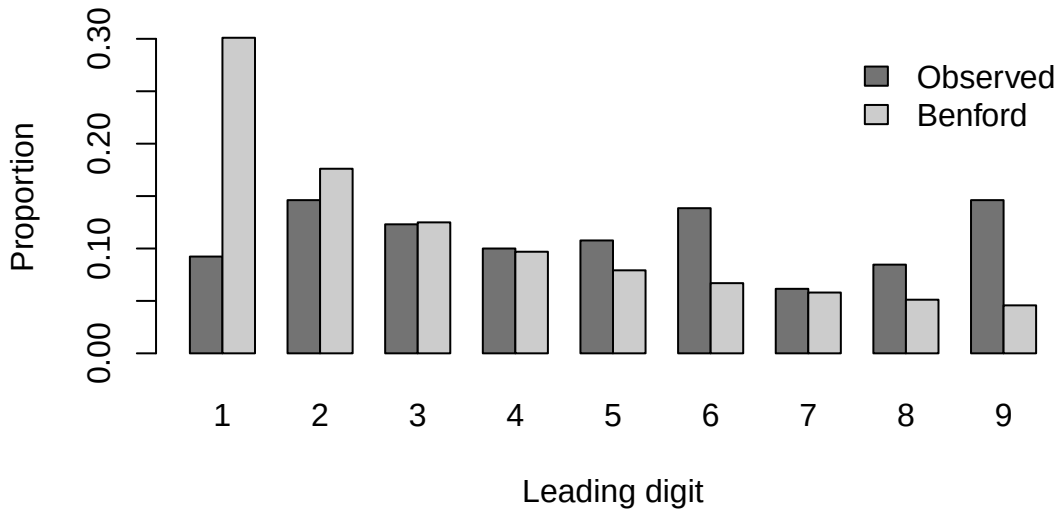
Digit	Count	Observed	Expected	Expected_count
1	12	0.092	0.301	39.13
2	19	0.146	0.176	22.89
3	16	0.123	0.125	16.24
4	13	0.100	0.097	12.60
5	14	0.108	0.079	10.29
6	18	0.138	0.067	8.70
7	8	0.062	0.058	7.54
8	11	0.085	0.051	6.65
9	19	0.146	0.046	5.95

```r
kable(data.frame(
  Chi_squared = unname(benford_test$statistic),
  df = unname(benford_test$parameter),
  p_value = format.pval(benford_test$p.value, digits = 3, eps = 0.001)
), digits = 3, caption = "Benford goodness-of-fit test")
```

Table 20: Benford goodness-of-fit test

Chi_squared	df	p_value
62.268	8	<0.001

```r
barplot(rbind(observed / sum(observed), benford), beside = TRUE,
        names.arg = 1:9, col = c("grey45", "grey80"),
        xlab = "Leading digit", ylab = "Proportion",
        legend.text = c("Observed", "Benford"),
        args.legend = list(x = "topright", bty = "n"))
```



The test rejects Benford's law at the 5% level. These apartment numbers do not pass a Benford screening. However, apartment numbers are assigned identifiers with restricted ranges, so real apartment numbers need not follow Benford's law. This test alone cannot establish whether the data are real; the dataset is described as synthetic.